Data Cleansing Approach

Project : Nusantara Car Motor – car dealer analytics (portfolio project, Data Analyst)

Platform : Google BigQuery & Microsoft Excel

Dataset : project-nusantara-car.dataset_extraction

Views(CTE) : clean_customers, clean_dealers, clean_vehicles, clean_transactions

Stage : Data Exploration – Data Cleansing – Data Validation – Data Analysis – Insight & Recommendation

1. Overview

1.1 Purpose and Scope

This document explains, one Common Table Expression (CTE) at a time, the four cleansing views that sit between the raw tables and the analysis layer. For every CTE it states what the CTE receives, what it changes, why the change is needed, and what the result looks like in numbers. The goal is that a reviewer can read the SQL and the document side by side and understand every decision.

1.2 Data Cleansing Pipeline Summary

View	Raw source	CTE chain	Rows in → out
clean_customers	customers	dataset_source_customers → standardized_first_approach → handling_duplicate_customer → final SELECT	10,690 → 10,690 (10,606 unique customers via master_customer_id)
clean_dealers	dealers	dataset_source_dealers → standarized_dealers_first_approach → final SELECT	32 → 32
clean_vehicles	vehicles	data_source_vehicle → standarized_vehicles_first_approach → final SELECT	62 → 62
Clean_transactions	transactions	data_source_transactions → standardized_first_approach_transactions → categorized_second_approach_for_csat_score → final SELECT	11,939 → 11,939

1.3 How each CTE section is laid out

Every CTE has a short summary table (purpose, input, output), the SQL as deployed, a “What it does” list, and, where a real decision was made, a highlighted “Key decision” note. CTE names are reproduced exactly as they appear in the deployed SQL (including the spelling standardized).

2. View: clean_customers

Primary data: one row per customer_id (10,690 rows).

Added column: master_customer_id.

Issues addressed: phone numbers stored as integers (leading zero lost), inconsistent casing and whitespace in names and cities, marketing_opt_in stored as 1/0, and the same person registered more than once.

CTE 1 dataset_source_customers

Purpose : Single, explicit entry point to the raw customers table; converts phone to text so string functions can be used.

Input : project-nusantara-car.dataset_extraction.customers (10,690 rows)

Output : 10,690 rows, 13 columns, phone is exposed as phone_text (STRING)

Query :

```
dataset_source_customers AS (
  SELECT
    customer_id, full_name, gender, birth_date, email,
    CAST(phone AS STRING) AS phone_text,
    city, province, occupation, monthly_income_idr,
    customer_segment, registration_date, marketing_opt_in
  FROM `project-nusantara-car.dataset_extraction.customers`
)
```

What it does

- Lists the 13 columns explicitly instead of using SELECT *, so a schema change in the raw table cannot silently flow into the analysis layer.
- CAST(phone AS STRING): when the CSV was loaded, BigQuery auto-detected phone as INT64, which drops the leading zero (0812… becomes 812…). The CSV file itself is intact. Casting to text is required before LIKE and CONCAT can be applied in the next CTE.
- The alias phone keeps the raw representation separate from the cleaned phone that is produced in CTE 2.

CTE 2 standardized_first_appraoch

Purpose : Standardise text, phone and flag formats so that records can be compared, grouped and joined reliably.

Input : dataset_source_customers

Output : 10,690 rows, 13 columns(phone_text replaced cleaned phone, marketing_opt_in now text)

Query :

```
standardized_first_approach AS (
  SELECT
    customer_id,
    CASE WHEN customer_segment = 'Retail' THEN INITCAP(TRIM(full_name))
        ELSE TRIM(full_name) END AS full_name,
    LOWER(TRIM(email)) AS email,
    CASE WHEN phone_text LIKE '8%' THEN CONCAT('62', phone_text)
        ELSE phone_text END AS phone,
    INITCAP(TRIM(city)) AS city,
    CASE marketing_opt_in WHEN 1 THEN 'Yes' WHEN 0 THEN 'No'
        ELSE 'Unknown' END AS marketing_opt_in
    -- gender, birth_date, province, occupation, monthly_income_idr,
    -- customer_segment, registration_date pass through unchanged
  FROM dataset_source_customers
)
```

Transformations and their effect on the data

Column	Rule	Observed effect
full_name	Retail: INITCAP(TRIM()). Corporate / Fleet: TRIM() only	43 Retail names stored in ALL CAPS become proper case (e.g. DIAN UTOMO → Dian Utomo). Company names keep their original form because INITCAP would damage abbreviations such as PT and CV.
email	LOWER(TRIM())	No rows needed changing in the current data, but the rule protects joins and duplicate detection against future loads. The 423 NULL emails stay NULL.
phone	If the text starts with 8, prefix 62; otherwise keep as is.	10,388 numbers gain the 62 prefix; 24 numbers already start with 62 and are left alone; 278 NULLs stay NULL. Every non-NULL value now matches ^628[0-9]{9,10}$.
city	INITCAP(TRIM())	149 spelling variants (JAKARTA SELATAN, Bandung, Surabaya …) collapse into 36 canonical cities; 330 rows changed, 173 of them because of stray spaces.
Marketing_opt_in	CASE: 1 → Yes, 0 → No, else Unknown	6,999 Yes, 3,691 No, 0 Unknown. Readable labels for charts and filters.
Unchanged columns	–	gender and birth_date have 948 NULLs; these are exactly the 724 Corporate and 224 Fleet customers, for whom the attributes do not apply. They are structural NULLs and are deliberately left empty.

Key decision – phone standard

Numbers that start with 8 get the country prefix 62 (international format without “+”), instead of removing the existing 62 from other rows. Reasons: the 8 prefix is by far the most common form, a prefix is simply added (no information is thrown away), and the result is one consistent format that also matches the 24 numbers that already had it.

The ELSE phone_text branch is intentional: a NULL phone makes NULL LIKE '8%' evaluate to NULL, falls into ELSE, and stays NULL. This matters in CTE 3, where a NULL phone must not be used as a duplicate key.

CTE 3 handling_duplicate_customer

Purpose : Detect people who were registered more than once and give every duplicate the customer_id of its "master" record, without deleting rows and without changing customer_id.

Input : standardized_first_approach.

Output : 10,690 rows, 14 columns (all standardised columns plus master_customer_id.

SQL Query :

```
handling_duplicate_customer AS (
  SELECT
    customer_id, full_name, gender, birth_date, email, phone, city,
    province, occupation, monthly_income_idr, customer_segment,
    registration_date, marketing_opt_in,
    CASE WHEN customer_segment = 'Retail' AND phone IS NOT NULL THEN
         FIRST_VALUE(customer_id) OVER (
           PARTITION BY full_name, phone
           ORDER BY (email IS NOT NULL) DESC, registration_date, customer_id)
        ELSE customer_id END AS master_customer_id
  FROM standardized_first_approach
)
```

How the logic works:

- **Duplicate key:** the cleaned full_name plus the cleaned phone. A phone number identifies a person far better than a name (many different people share a name), and the name is compared after CTE 2 has normalised casing and spaces, so DIAN UTOMO and Dian Utomo match.
- **Retail only.** Corporate and Fleet customers have no birth_date, and the 184 company names that appear twice were examined and are coincidences (176 of the 184 groups have different phone numbers; registrations are on average 797 days apart), not duplicates. They always keep master_customer_id = customer_id.
- **NULL protect.** The condition phone IS NOT NULL is needed because PARTITION BY treats all NULLs as one group. Without the guard, Retail customers who have no phone number would be merged whenever they share a name, although there is no evidence they are the same person (250 Retail customers have no phone; one same-name pair exists among them today).
- **Choosing the master.** ORDER BY (email IS NOT NULL) DESC, registration_date, customer_id ranks the record that has an email first (the more complete one), then the earliest registration, then the lowest ID as a deterministic tie-breaker. FIRST_VALUE returns that record's customer_id for every row in the group.
- **Worked example from the data**

customer_id	full_name	phone	email	registration_date	master_customer_id
C00314	Dewi Lim	628223598594	dewi.lim77@icloud.com	2021-03-30	C00314
C04942	Dewi Lim	628223598594	NULL	2023-12-07	C00314

Result:

- **84 duplicate groups**, each with exactly two records, so 84 rows point to a different master.
- **10,606 unique customers** (10,690 − 84). Use COUNT(DISTINCT master_customer_id) for any "number of customers" metric.
- **Transactions are untouched.** 56 transactions still reference a non-master customer_id. To aggregate by person, join clean_transactions to clean_customers on customer_id and group by master_customer_id. Among customers who bought, this turns 9,809 distinct customer_id values into 9,753 real customers.
- **Deliberately not merged:** pairs that share a name but where one record has only a phone and the other only an email. There is not enough evidence to call them the same person, so they stay separate.

**Note:*

Key decision – soft merge instead of delete

Deleting or re-pointing duplicate IDs would break the link to transactions and would hide the fact that a customer re-registered (itself a useful business signal). A window-function flag keeps every raw fact, keeps all 11,939 transaction rows joinable, and still gives analysts one reliable customer key.

CTE 4 Final SELECT

Purpose : Expose the finished column list, in a deliberate order, as the view's output.

Input : handling_duplicate_customer.

Output : clean_customers: 10,690 rows x 14 columns.

Query :

```
SELECT
  customer_id, master_customer_id, full_name, gender, birth_date,
  email, phone, city, province, occupation, monthly_income_idr,
  customer_segment, registration_date, marketing_opt_in
FROM handling_duplicate_customer;
```

- master_customer_id is placed directly after customer_id so the pair is easy to spot.
- phone_text no longer appears; only the cleaned phone is exposed. occupation is included and passed through unchanged.

3. View: clean_dealers

Primary data: one row per dealer_id (32 rows).

Issues addressed: the phone number has the same integer problem as customers (stored as 9 digits, leading zero lost) and is_active is a 1/0 flag.

CTE 1 dataset_source_dealers

Purpose : Explicit entry point to the raw dealers table; converts phone to text.

Input : project-nusantara-car.dataset_extraction.dealers (32 rows)

Output : 32 rows, 14 columns; phone is now STRING.

Query :

```
dataset_source_dealers AS (
  SELECT
    dealer_id, dealer_name, dealer_type, city, province, region, address,
    CAST(phone AS STRING) AS phone,
    manager_name, opened_date, closed_date, is_active,
    monthly_sales_target_units, showroom_area_sqm
  FROM `project-nusantara-car.dataset_extraction.dealers`
)
```

- Unlike the customers view, the alias stays phone because the CTE that follows overwrites it with the standardised value.

CTE 2 standarized_dealers_first_approach

Purpose : Apply same phone standard as customers and give is_active readable labels.

Input : dataset_source_dealers.

Output : 32 rows, 14 column.

Query :

```
standarized_dealers_first_approach AS (
  SELECT
    dealer_id, dealer_name, dealer_type, city, province, region, address,
    CASE
      WHEN phone NOT LIKE '62%' THEN CONCAT('62', phone)
      ELSE phone
    END AS phone,
    manager_name, opened_date, closed_date,
    CASE is_active WHEN 1 THEN 'Active' WHEN 0 THEN 'Non Active'
        ELSE 'Unknown' END AS is_active,
    monthly_sales_target_units, showroom_area_sqm
  FROM dataset_source_dealers
)
```

Column	Rule	Observed effect
Phone	If the number does not already start with 62, prefix 62.	All 32 numbers go from 9 digits to 11 digits starting with 62 (e.g. 657194391 → 62657194391). Dealer numbers are landlines with an area code; the standard is the same as for customers: country code, no leading zero.
Is_active	CASE: 1 → Active, 0 → Non Active, else Unknown	31 Active, 1 Non Active (D024, closed on 2024-11-30).
Unchanged columns	–	closed_date is NULL for the 31 active dealers. That NULL is structural (“not closed”) and is intentionally not filled.

**Note:*

The rule tests NOT LIKE '62%' rather than LIKE '8%' because dealer numbers begin with area-code digits, not with 8. In the current data no dealer number starts with 62 once the leading zero is lost, so every number receives the prefix.

CTE 3 Final SELECT

Purpose : Expose the finished column list.

Input : standardized_dealers_first_approach

Output : clean_dealers: 32 rows x 14 columns

Query :

```
SELECT
  dealer_id, dealer_name, dealer_type, city, province, region, address,
  phone, manager_name, opened_date, closed_date, is_active,
  monthly_sales_target_units, showroom_area_sqm
FROM standarized_dealers_first_approach;
```

4. View: clean_vehicles

Primary data: one row per vehicle_id(62 rows).

Brief Description: The vehicle master was the cleanest table in the inspection: no duplicates, no formatting problems in text columns, and the only NULL column is discontinued_date (56 NULLs = models still on sale, a structural NULL). The only standardisation needed is the is_active flag.

CTE 1 data_source_vehicle

Purpose : Explicit entry point to the raw vehicles table.

Input : project-nusantara-car.dataset_extraction.vehicles (62 rows)

Output : 62 rows, 16 columns, unchanged.

Query :

```
data_source_vehicle AS (
  SELECT
    vehicle_id, brand, model, variant, body_type, fuel_type,
    transmission, engine_cc, seating_capacity, assembly_origin,
    model_year, list_price_idr, cost_idr, launch_date,
    discontinued_date, is_active
  FROM `project-nusantara-car.dataset_extraction.vehicles`
)
```

- cost_idr is kept here, in the vehicle master, where it belongs. Margin analysis reaches it with a LEFT JOIN from transactions on vehicle_id; it is not copied into clean_transactions.

CTE 2 standarized_vehicles_first_approach

Purpose : Give is_active readable labels; leave every other column untouched.

Input : data_source_vehicle.

Output : 62 rows, 16 columns.

Query :

```
standarized_vehicles_first_approach AS (
  SELECT
    -- vehicle_id … discontinued_date: unchanged
    CASE WHEN is_active = 1 THEN 'Active'
         WHEN is_active = 0 THEN 'Inactive'
         ELSE 'Unknown' END AS is_active
  FROM data_source_vehicle
)
```

- 56 vehicles are Active and 6 are Inactive.
- Edge cases reviewed during inspection and kept as valid: two 2027 model-year vehicles (V061, V062) have launch dates in November 2026, i.e. they are pre-launch models; two transactions (T006344, T007658) fall on the exact discontinued_date of their vehicle, which is the last day of sale. No transaction occurs before a launch date or after a discontinuation date.

CTE 3 Final SELECT

Purpose : Expose the result

Input : standarized_vehicles_first_approach

Output : 62 rows, 16 columns.

Query :

```
SELECT * FROM standarized_vehicles_first_approach
```

- SELECT * is acceptable here because the previous CTE already defines the exact column list; nothing extra can slip through.

5. View: clean_transactions

Primary data: one row per transaction_id(11,939 rows). The view deliberately reads from one table only (no joins to customers, vehicles or dealers), so it stays simple and fast; joins are performed in the analysis layer.

Issues addressed: NULL text labels, 1/0 flags, and a satisfaction score that is NULL for more than half of the rows.

CTE 1 data_source_transactions

Purpose : Explicit entry point to the raw transactions table.

Input : project-nusantara-car.dataset.extraction.transactions (11,939 rows)

Output : 11,939 rows, 20 columns, unchanged.

Query :

```
data_source_transactions AS (
  SELECT
    transaction_id, transaction_date, customer_id, vehicle_id, dealer_id,
    salesperson_name, quantity, unit_price_idr, discount_idr,
    total_amount_idr, payment_method, financing_partner,
    down_payment_pct, tenor_months, lead_source, is_trade_in,
    insurance_included, status, delivery_date, csat_score
  FROM `project-nusantara-car.dataset_extraction.transactions`
)
```

CTE 2 standardized_first_approach_transactions

Purpose : Replace NULL labels with meaningful text and convert 1/0 flags to Yes / No.

Input : data_source_trasncactions

Output : 11,939 rows, 20 columns.

Query :

```
standardized_first_approach_transactions AS (
  SELECT
    transaction_id, transaction_date, customer_id, vehicle_id, dealer_id,
    COALESCE(salesperson_name,'Unknown') AS salesperson_name,
    quantity, unit_price_idr, discount_idr, total_amount_idr, payment_method,
    COALESCE(financing_partner,'Not Used') AS financing_partner,
    down_payment_pct, tenor_months,
    COALESCE(lead_source,'Unknown') AS lead_source,
    CASE is_trade_in WHEN 1 THEN 'Yes' WHEN 0 THEN 'No'
        ELSE 'Unknown' END AS is_trade_in,
    CASE insurance_included WHEN 1 THEN 'Yes' WHEN 0 THEN 'No'
        ELSE 'Unknown' END AS insurance_included,
    status, delivery_date, csat_score
  FROM data_source_transactions
)
```

Column	Rule	Observed effect
salesperson_name	COALESCE(…, 'Unknown')	164 NULLs become Unknown. The name is genuinely missing, so the honest label is “Unknown” rather than a guessed manager.
financing_partner	COALESCE(…, 'Not Used')	3,868 NULLs become Not Used. All of them are Cash payments, so the NULL is structural (no loan, no partner). Not Used is used instead of Unknown because the absence is expected, not missing.
lead_source	COALESCE(…, 'Unknown')	331 NULLs become Unknown (the sales record did not capture how the customer arrived).
is_trade_in	CASE: 1 → Yes, 0 → No, else Unknown	1,434 Yes, 10,505 No, 0 Unknown.
insurance_included	CASE: 1 → Yes, 0 → No, else Unknown	7,204 Yes, 4,735 No, 0 Unknown.
Unchanged columns	–	down_payment_pct and tenor_months are NULL on the same 3,868 Cash rows and stay NULL (0% down payment or 0 months would distort averages). delivery_date is NULL for 969 rows, exactly the 760 Cancelled + 209 Pending deals that were never delivered. csat_score is handled in CTE 3.

Note: why COALESCE for labels

COALESCE(col, label) replaces only NULLs and passes every populated value through untouched, which is exactly the intent for text columns. CASE is used where a value must be translated (1/0 → Yes/No). Both are SELECT logic: the raw table is never modified.

CTE 3 categorized_second_approach_for_csat_score

Purpose : Create a readable satisfaction category while keeping csat_score honest (NULL stays NULL).

Input : standardized_first_approach_transactions

Output : 11,939 rows, 21 columns, 1 columns was added (csat_category) based on csat_score.

Query :

```
categorized_second_approach_for_csat_score AS (
  SELECT
    *,
    CASE WHEN csat_score >= 4 THEN 'Satisfied'
        WHEN csat_score = 3  THEN 'Netral'
        WHEN csat_score <= 2 THEN 'Dissatisfied'
        WHEN status IN ('Pending', 'Cancelled') THEN 'N/A'
        ELSE 'No Response' END AS csat_category
  FROM standardized_first_approach_transactions
)
```

csat_category	Rule	Rows
Satisfied	csat_score 4 or 5	4,551
Netral	csat_score = 3	873
Dissatisfied	csat_score 1 or 2	625
N/A	No score and status is Pending or Cancelled (the deal never reached delivery, so no survey was possible)	969
No Response	No score on a delivered deal (Completed or Returned): the customer did not answer the survey	4,921

How the CASE works

- Order matters. CASE evaluates top to bottom and stops at the first true condition. The three numeric tests run first, so any real score is classified immediately.
- NULL scores fall through. A comparison such as NULL >= 4 is not TRUE, so a NULL score skips all three numeric branches and reaches the status test, then the ELSE.
- Scores are never replaced by 0. csat_score stays NULL, so averages and score distributions are calculated on real answers only. The separate No Response category turns the gap into an insight: among Completed sales, 5,836 of 10,655 customers (54.8%) answered, so a survey-incentive campaign has clear room to improve the response rate.
- SELECT * here simply carries all 20 columns of the previous CTE forward and adds the new one.

CTE 4 Final SELECT

Purpose : Expose the result

Input : categorized_second_approach_for_csat_score

Output : clean_transactions: 11,939 rows x 21 columns

Query :

```
SELECT * FROM categorized_second_approach_for_csat_score;
```

Usage notes for the analysis layer

- *Revenue is Completed sales only (status = 'Completed'). Completed deals total Rp 5,611,376,200,000. Pending deals are "Booked Sales (Pending)", not revenue; Cancelled and Returned are excluded. The view keeps every status so that these rules are applied, and can be changed, in the analysis queries.*
- *Customer-level metrics must use master_customer_id from clean_customers (see section 2, CTE 3).*
- *Margin analysis joins clean_vehicles on vehicle_id to obtain cost_idr.*

Appendinx

Validation results

The four views were executed against the CSV source data in BigQuery and for customers tables (phone field loaded as a 64-bit integer to reproduce BigQuery's INT64 auto-detection). The same checks should be re-run once in BigQuery after deployment.

Check	Expected	Observed	Result
Row counts (customers / dealers / vehicles / transactions)	10,690 / 32 / 62 / 11,939	10,690 / 32 / 62 / 11,939	Pass
Raw vs clean SUM(total_amount_idr)	Identical	6,299,981,200,000 both	Pass
Completed revenue unchanged	Identical	5,611,376,200,000 both	Pass
Duplicate groups; unique customers	84; 10,606	84; 10,606	Pass
Corporate / Fleet rows merged	0	0	Pass
Every master is its own master (no chains)	0 violations	0 violations	Pass
Customer phone matches ^628[0-9]{9,10}$; NULLs retained	0 violations; 278 NULL	0 violations; 278 NULL	Pass
Dealer phone matches ^62[0-9]{8,11}$	0 violations	0 violations	Pass
Label sets (Yes/No, Active/Non Active, Active/Inactive)	No Unknown	No Unknown	Pass
NULL fills: salesperson / lead source / financing	164 / 331 / 3,868	164 / 331 / 3,868	Pass
csat_category counts	4,551 / 873 / 625 / 4,921 / 969	4,551 / 873 / 625 / 4,921 / 969	Pass
Orphan keys: transactions → customers / vehicles / dealers	0 / 0 / 0	0 / 0 / 0	Pass
Join fan-out: transactions ⋈ clean_customers	11,939 rows	11,939 rows	Pass

Optional refinements

None of these affect correctness or the numbers above. They are consistency and robustness improvements to consider.

No	Observation	Suggestion
1	csat_category uses the Indonesian word Netral next to English Satisfied and Dissatisfied.	Rename to Neutral for a fully English output.
2	Inactive label differs: dealers use Non Active, vehicles use Inactive.	Use Inactive in both views.
3	The spelling standarized appears in two CTE names; standardized is used elsewhere.	Cosmetic only. Rename if you wish, keeping every reference consistent.
4	In clean_dealers, the FROM clause of CTE 2 wraps the CTE name in backticks. BigQuery accepts this.	Remove the backticks to match the other views.
5	The dealer rule NOT LIKE '62%' would treat a future number whose area code begins with 62 as already prefixed.	Optionally add a length test; not needed for the current 32 dealers.
6	The customer rule only prefixes numbers that start with 8. Any future number with a different start would pass through unprefixed.	Keep the phone-format check from Appendix A as a recurring test.

Label dictionary

View · column	Raw value	Cleaned value
clean_customers · marketing_opt_in	1 / 0 / NULL	Yes / No / Unknown
clean_dealers · is_active	1 / 0 / NULL	Active / Non Active / Unknown
clean_vehicles · is_active	1 / 0 / NULL	Active / Inactive / Unknown
clean_transactions · is_trade_in, insurance_included	1 / 0 / NULL	Yes / No / Unknown
clean_transactions · salesperson_name, lead_source	NULL	Unknown
clean_transactions · financing_partner	NULL (Cash deals)	Not Used
clean_transactions · csat_category (new)	csat_score and status	Satisfied, Netral, Dissatisfied, N/A, No Response
clean_customers · phone	8… (leading zero lost); 62…; NULL	628…; unchanged; NULL
clean_dealers · phone	9 digits (leading zero lost)	62 + 9 digits